

Core Implementation of Bhutan Business Registration Chatbot

Embedding Model

BAAI/bge-base-en-v1.5

LLM Model

Llama 3.1 8B Instant

through
Groq

Vector Database

Chroma

Conversational Flow

1. User Starts Conversation

User enters:

- Text input
or
- Voice input

2. Language Selection

User selects:

- English
- Dzongkha

If Dzongkha is selected:

- Input is translated into English using deep-translator

3. Input Processing

The chatbot checks:

- Greeting
- Abusive language
- Invalid input
- Intent detection

Example intents:

- New business registration
- License renewal

4. Conversational Flow Starts

If user wants new business registration:

Chatbot asks step-by-step:

1. Citizenship
2. Age
3. Business type
4. Location

If user wants renewal:

Chatbot asks:

1. Business type
2. License status

5. Context Retrieval (RAG)

The user question is converted into embeddings using
BAAI/bge-base-en-v1.5

Then relevant PDF chunks are retrieved from
Chroma

6. LLM Response Generation

Retrieved context + conversation history + user question are sent to

Llama 3.1 8B Instant

through
Groq

The model generates the final response.

7. Chat History Storage

Conversation is stored using:

- Session ID
- User messages
- Assistant responses

8. Voice Output

If voice mode is enabled:

- Response is converted into speech using
pyttsx3

9. Continuous Conversation

The chatbot continues:

- Using conversation history
- Maintaining flow state

- Asking one question at a time
 - Handling topic changes naturally
-

Streamlit User Interface Workflow

Streamlit Workflow

1. Load Environment Variables

- Load `.env`
- Read GROQ API key

2. Initialize GROQ Client

- Connect chatbot with Groq

3. Configure Streamlit Page

- Set title
- Set icon
- Set layout

4. Audio Transcription

- User records voice
- Audio sent to Whisper
- Voice converted into text

5. Text-to-Speech

- Assistant response converted into voice using edge-tts

6. Session State Initialization

Store:

- Chat state
- Session ID
- Messages
- Audio cache
- Language
- Input mode

7. Sidebar Features

Sidebar contains:

- New chat
- Language selection
- Chat history
- Delete chat option

8. Chat Display

- Show user messages
- Show assistant responses
- Read response button
- Stop audio button

9. Input Handling

Two input modes:

- Text input
- Voice input

10. Voice Workflow

- Record audio
- Transcribe speech into text
- Show transcribed text
- User confirms message

11. Translation Layer

If Dzongkha selected:

- Translate Dzongkha → English using deep-translator

12. Chatbot Processing

User message sent to:

- `controller()`
- Conversational flow
- RAG pipeline
- LLM response generation

13. Save Chat History

Store:

- User message
- Assistant response
- Session ID

Chat Storage Database : Supabase

Purpose

Used for:

- Saving chat history
- Loading previous sessions
- Deleting chats
- Managing conversation sessions

14. Continuous Conversation

- Maintain conversation state
- Load previous chats

- Continue old sessions naturally

15. Footer Disclaimer

Display:

- Verification warning
 - Official Bhutan source notice
-

Problems Faced During Development

Limitations of Mistral Small Model

Initially, I used
Mistral AI small model for the chatbot.

Some problems faced were:

- The model struggled to manage multiple conversational flows correctly.
- It sometimes failed to understand whether the user was continuing a conversation or asking a new RAG-related question.
- Memory-based conversations became inconsistent.
- Responses were sometimes confusing during long conversations.

Memory System Challenge

I implemented a memory system for storing and recalling previous conversation context.

However, during testing:

- When users asked questions related to the conversational flow, the chatbot sometimes triggered the conversational flow again instead of retrieving information from the RAG system.
- The model could not properly differentiate between:
 - conversational flow questions
 - normal RAG information queries

This created confusion in the chatbot behavior.

User Experience Issue

For rural or less technical users:

- The memory-based conversational behavior became difficult to understand.
- Users expected direct answers, but sometimes the chatbot continued old flows unexpectedly.
- This made the interaction more complex and less user-friendly.

Because of this, I removed the memory-based flow to keep the chatbot:

- simpler
- clearer
- easier to use
- more reliable for users.

Voice Model Performance Challenge

During development and in the last project meeting, I used

Whisper Large V3

for the voice input system.

Some challenges faced were:

- The model required higher processing power.
- Voice transcription sometimes took more time during real-time interaction.
- Performance depended on internet speed and API response time.
- Continuous voice interaction could become slower during longer conversations.

Because of this, optimizing voice performance became an important part of the development process.

How the Challenges Were Overcome

LLM Optimization

Using
Groq

Solved:

- Faster response generation
- Lower latency
- Better real-time conversation performance
- Smoother chatbot interaction

Voice Processing Optimization

Using
Whisper Large V3 Turbo

Solved:

- Reduced voice transcription time
 - Faster speech-to-text conversion
 - Improved user experience
-

Plan for Phase 2

Dzongkha Language Expansion

Currently, the chatbot supports:

- Dzongkha text input
- Dzongkha to English translation

Future improvements needed:

- Dzongkha voice input
- Dzongkha text output

- Dzongkha voice output

Possible APIs and services:

- Google Cloud Translation API
- Microsoft Azure Speech Services
- OpenAI Whisper API
- ElevenLabs TTS API

These APIs can help provide:

- Better Dzongkha translation
- Speech recognition
- Natural voice responses

Streamlit Improvements

Future improvements for
Streamlit

- Better UI/UX design
- Faster loading performance
- Real-time response streaming
- Better voice interaction handling
- Improved mobile responsiveness
- Better chat state management

Cloud Deployment Plan

Free Cloud Options

Possible free deployment platforms:

- Streamlit Community Cloud
- Render
- Hugging Face Spaces

Limitations:

- Limited resources
- Sleep mode after inactivity
- Slower performance for heavy AI workloads

Paid Cloud Options

Possible paid cloud services:

- Amazon Web Services (AWS)
- Google Cloud Platform (GCP)
- Microsoft Azure

Benefits:

- Better scalability
 - Faster processing
 - 24/7 availability
 - Better support for AI applications
 - More stable real-time voice interaction
-

Additional Improvements

Authentication System

Add:

- User login/signup
- Secure user sessions
- Personalized chat history

Possible services:

- Supabase Authentication
- Firebase Authentication

Admin Dashboard

Future admin features:

- Monitor chatbot usage
- View analytics
- Manage uploaded PDF documents
- Track user queries and feedback

API Integration

Possible future integrations:

- Bhutan government business portals
- License verification systems
- Real-time status checking APIs

Scalability Improvements

Future scalability plans:

- FastAPI backend integration
- Separate frontend and backend architecture
- Load balancing
- GPU-based deployment for faster AI inference

Security Improvements

Future security features:

- API rate limiting
- Encrypted chat storage
- Secure environment variable handling
- User access control

Analytics and Feedback System

Add:

- User feedback collection
- Response rating system
- Error monitoring
- Performance tracking

Mobile Application

Possible future development:

- Android application
 - iOS application
 - Voice-first mobile assistant experience
-